Ergebnisse

XOR, 7-Segment-Decoder und ein lokales Minimum

Inhaltsverzeichnis

Interaktive Visualisierung 1
XOR 1
Was das Netz gelernt hat 2
Lokales Minimum 2
7-Segment-Decoder 2
Export und Import 3

Alle Zahlen stammen aus Läufen der Python-Version mit den Parametern der Angabe: Sigmoid, Lernrate $\alpha = 0,1$, 20 000 Epochen, Startgewichte gleichverteilt in $[-1, 1)$ mit festem Seed. Die Kosten sind der Mittelwert von J über alle Beispiele einer Epoche.

Interaktive Visualisierung

Die Visualisierung zeigt das Training Schritt für Schritt: Netzgraph mit Gewichten und Aktivierungen, Kostenverlauf, die Entscheidungsfläche beim XOR und die 7-Segment-Anzeige. Alle Werte wurden während des Trainings aus der Python-Version aufgezeichnet; die Seite rechnet selbst nichts.

XOR

Netz 2-2-1, Seed 0, Laufzeit etwa 1,4 Sekunden.

Epoche	Kosten
0	0,1315
2 000	0,1253
4 000	0,0991
6 000	0,0100
10 000	0,0017
20 000	0,0005

Die ersten rund 4 000 Epochen passiert wenig: Das Netz gibt für alle Inputs etwa 0,5 aus. Dann findet es die Lösung, und die Kosten fallen innerhalb von 2 000 Epochen um eine Zehnerpotenz.

Input	Soll	Ausgabe	gerundet
[0, 0]	0	0,0343	0 ✓
[0, 1]	1	0,9699	1 ✓

Input	Soll	Ausgabe	gerundet
[1, 0]	1	0,9708	1 ✓
[1, 1]	0	0,0308	0 ✓

Was das Netz gelernt hat

Die Gewichte (siehe Dateiformate) lassen sich direkt deuten:

Neuron	gewichtete Summe	aktiv bei
Hidden 1	$5,90\,x_1 - 5,91\,x_2 - 3,36$	$x_1 = 1$ und $x_2 = 0$
Hidden 2	$-5,42\,x_1 + 5,14\,x_2 - 2,86$	$x_1 = 0$ und $x_2 = 1$
Output	$8,16\,h_1 + 8,31\,h_2 - 4,06$	h_1 oder h_2 aktiv

Das Netz hat XOR also selbst in „(x_1 und nicht x_2) oder (x_2 und nicht x_1)„ zerlegt — genau die Form, die man auch von Hand aufschreiben würde.

Lokales Minimum

Ob das kleine XOR-Netz die Lösung findet, hängt von den Startgewichten ab. Mit den Seeds 0 bis 19 und sonst gleichen Parametern:

Ergebnis	Seeds	Anzahl
alle 4 Fälle richtig, Kosten ≈ 0,0005	alle übrigen	16
hängen geblieben, Kosten ≈ 0,064	8, 12, 13	3
hängen geblieben, Kosten ≈ 0,085	15	1

Bei Seed 8 lernt das Netz $[0, 0] \to 0,03$ und $[0, 1] \to 0,96$, gibt aber für $[1, 0]$ und $[1, 1]$ beide Male etwa 0,50 aus. Eine Ausgabe von 0,5 ist für zwei Inputs mit unterschiedlichem Soll der beste Kompromiss, den dieses Netz von dort aus erreichen kann — der Gradient ist praktisch null, auch nach 20 000 Epochen. In der Visualisierung ist das als eigenes Szenario zu sehen.

Mehr Hidden-Neuronen, ein anderer Seed oder mehrere Läufe, von denen man den besten behält. Für die Abgabe verwenden wir Seed 0.

7-Segment-Decoder

Netz 4-16-7, Seed 42, Laufzeit etwa 18 Sekunden. Alle **10 von 10 Ziffern** werden richtig dekodiert.

Epoche	Kosten
0	1,326
2 000	0,0129
4 000	0,0040
10 000	0,0011
20 000	0,0005

Mit 16 Hidden-Neuronen gibt es kein langes Plateau wie beim XOR; die Kosten fallen von Anfang an.

Ziffer	Input	Soll a–g	Ausgabe a–g
0	0000	1111110	0,98 1,00 0,99 0,99 0,99 0,99 0,02
1	0001	0110000	0,02 1,00 1,00 0,01 0,00 0,02 0,00
2	0010	1101101	1,00 1,00 0,02 0,99 1,00 0,01 0,98
3	0011	1111001	1,00 0,99 0,99 0,99 0,01 0,00 0,98
4	0100	0110011	0,02 0,98 1,00 0,02 0,01 1,00 0,98
5	0101	1011011	0,98 0,02 1,00 0,98 0,00 0,99 0,98
6	0110	1011111	1,00 0,02 0,99 0,99 0,99 0,99 1,00
7	0111	1110000	0,98 0,99 1,00 0,02 0,00 0,00 0,03
8	1000	1111111	1,00 1,00 1,00 1,00 0,99 1,00 1,00
9	1001	1111011	0,99 0,99 1,00 1,00 0,00 0,99 0,99

Jede Ausgabe liegt höchstens 0,03 vom Sollwert entfernt.

Export und Import

Beide Netze wurden als JSON, TXT und C-Header exportiert und aus JSON und TXT neu geladen. Die größte Abweichung zwischen Original und geladenem Netz über alle Inputs ist **0** — die Vorhersagen sind bitgleich.

Die Ergebnisse der C#-Version und der Vergleich zwischen beiden Sprachen kommen hier dazu, sobald die C#-Umsetzung fertig ist.